Evaluating Inference-Time Defences Against Jailbreak Attacks

Adam Kaegi
a2kaegi@uwaterloo.ca

Ryan Maxin
rsmaxin@uwaterloo.ca

Vishesh Gupta
v84gupta@uwaterloo.ca

Varun Mathur
v3mathur@uwaterloo.ca

I. Introduction and Motivation

Large Language Models (LLMs) are now at the core of a growing range of applications, yet they remain vulnerable to jailbreak attacks: carefully constructed inputs that push a model into compliance with a harmful or disallowed request. The attack surface is broad and evolving, spanning adversarial suffix attacks such as GCG [1] and automated prompt refinement methods such as PAIR [2], among others. Since deployed models are often treated as fixed inference endpoints, and retraining or modifying their internal representations may be impractical, inference-time defences are attractive because they can wrap an unmodified model with input filtering, prompt augmentation, generation-time intervention, or output vetting.

Most published defences, however, are evaluated narrowly against the attacks they were designed to address, motivating our research. Take perplexity filters for example. They detect anomalous or obfuscated text [3], but fluent attacks can bypass purely statistical defences. This motivates evaluating different defence mechanisms under the same attacks, target model, and evaluation procedure, as well as testing whether combining complementary defences provides better protection than any single defence alone.

II. Problem Statement

This project evaluates inference-time defences against a diverse set of public jailbreak attacks. We focus on three main research questions:

1. **Generalization:** Does each defence work across different kinds of jailbreaks, or does it only stop the narrow attack family it was built to address?
2. **Defence-in-Depth:** When we combine defences into a stack, does that stack block more attacks than the best individual defence, or does it mainly add redundant protection and additional cost?
3. **Cost:** For each defence and defence stack, what practical costs accompany improved security? We measure how often benign requests are incorrectly refused or blocked, how much response latency is added, and how many target-model calls are required.

The goal is not to invent new jailbreaks or defences. We evaluate existing public attacks, defences, and models under one controlled setup so that their security, usability, and performance trade-offs can be compared directly.

III. Related Work

Adversarial vulnerabilities in LLMs span both optimization-based attacks and semantic prompt engineering. Zou et al. introduced GCG, a white-box attack that uses model gradients to find adversarial token suffixes capable of disrupting alignment [1]. Chao et al. proposed PAIR, a black-box method in which an adversarial LLM iteratively rewrites a prompt using feedback from the target model [2]. DeepInception instead uses nested fictional scenarios and role-play to encourage the model to continue harmful behaviour through multiple layers of context [4]. More recent automated attacks include Tree of Attacks with Pruning (TAP), which extends iterative prompt search through branching and pruning [5], and PAPILLON, which applies automated fuzzing to generate concise jailbreak prompts [6]. These methods illustrate the range between token-level adversarial manipulation and fluent semantic attacks.

To standardize evaluation of these threats, JailbreakBench provides a reproducible benchmark containing harmful and benign behaviours together with public jailbreak artifacts [7]. HarmBench similarly provides standardized harmful behaviours and automated evaluation methods for testing whether jailbreak attacks successfully bypass a model's safety protections [8]. These benchmarks support our evaluation of four attack categories: direct harmful requests, nested role-play attacks, adversarial suffix attacks, and adaptive prompt refinement.

Inference-time defences mitigate these attacks at different points in the execution pipeline. For input filtering, Alon and Kamfonas show that perplexity can identify statistically unusual text such as machine-optimized adversarial suffixes [3]. Xie et al. propose Self-Reminder, a prompt-based defence that reinforces safety constraints before generation [9]. At the generation stage, Robey et al. introduce SmoothLLM, which perturbs the input and selects among multiple generations to reduce the effectiveness of fragile adversarial prompts [10]. For semantic safety filtering, Inan et al. present Llama Guard, which can classify both prompts and generated responses against a safety policy [11].

Other recent defences operate directly on internal model representations. JBSHIELD detects and manipulates hidden representations associated with jailbreak behaviour [12], while DEEPALIGN intervenes in hidden-state trajectories to steer generation back toward safer representations [13]. These methods require internal activation access and therefore fall outside our black-box inference-time threat model. Our study instead focuses on wrapper-based defences that can be applied without modifying the target model's weights or hidden states, and evaluates whether combining these mechanisms provides better protection than using them individually.

IV. Methodology

We evaluate inference-time defences against jailbreak attacks using a reproducible local evaluation harness, described in the Evaluation Harness section. We model the target as a black-

box LLM service: the attacker can submit prompts and observe responses, while the defender can only intervene at inference time through prompt transformation, additional model calls, early blocking, or output filtering. The target model's weights and internal representations remain unchanged.

Each harmful trial is defined by one target model, one benchmark behaviour, one attack condition, and one defence condition. Our four attack conditions are the raw benchmark behaviour, DeepInception [4], a transfer-based GCG attack [1], and a bounded PAIR-style adaptive attack [2]. These represent direct harmful requests, nested role-play prompts, adversarial suffixes, and adaptive prompt-generation attacks. We rely on public JailbreakBench inputs and attack artifacts [7] rather than manually creating new harmful objectives. For GCG and PAIR, the attacked prompt is constructed once for each harmful behaviour and reused unchanged across all defence conditions.

The experiment contains ten defence conditions: a no-defence baseline, four standalone defences, and five selected defence stacks. The standalone defences are Self-Reminder [9], perplexity filtering [3], SmoothLLM [10], and Llama Guard output filtering [11]. These intervene at different stages of the inference pipeline, while the selected stacks combine complementary mechanisms to test whether defence-in-depth improves robustness beyond the strongest individual defence.

Our evaluation has three stages:

1. **Baseline:** run each attack against the unprotected model to establish its initial attack success and response cost.
2. **Standalone defences:** test each individual defence against every attack type to determine which mechanisms generalize across attacks and which remain specialized.
3. **Defence stacks:** evaluate selected combinations of complementary defences under the same conditions to determine whether layering provides additional protection or primarily adds cost.

We also evaluate benign JailbreakBench prompts under every defence condition to measure usability alongside security. Our primary security metric is Attack Success Rate (ASR), determined using HarmBench [8]. WildGuard [14] separately measures refusal on harmful prompts and false refusal on benign prompts. We additionally measure response latency and target-model call count to capture the computational cost of each defence and stack. For GCG and PAIR, attack-construction time and query counts are reported separately from the defended response path.

V. Implementation

V.I. Evaluation Harness

The evaluation harness is our shared infrastructure enabling the evaluations in our methodology. It is a composable pipeline built with Python and LangChain that treats each component as interchangeable behind a common interface. The pipeline is as follows:

Prompt → **Attack** → Input-stage defence → **Model** → Output-stage defence → **Judge**

Each trial of the pipeline assembles a LangChain Expression Language (LCEL) chain that runs a prompt through the attack, any input-stage defences, the target model, and any output-stage defences. Separating input-stage from output-stage lets the harness place each defence at its correct point in the execution path: for example perplexity filtering [3] acts on the input, Llama Guard [11] can act on the generated response (output). The judge is run afterwards to deem the model output as either safe or unsafe. We time latency through the pipeline from the start of the input-defences stage to the end of the output-defences stage.

Just as each stage of the pipeline is, attacks are pluggable modules. Each attack lives in its own file, builds on a base class and is exposed through a shared registry. Adding a new attack simply involves adding a file to perform that attack, registering it, and then using it at runtime. This design enables us to swap attacks (and other similar components) for testing and trials without additional bespoke wiring.

Defences are implemented the same way with two key differences. First, each defence must either declare itself as an input-stage or output-stage defence so the pipeline knows at what point it should be applied. Second, as one of our goals is to test a stack of defences, defences can also be stacked within the pipeline and the order they are applied is the comma-separated order provided at runtime.

Target models are served locally through Ollama and selected by configuration, so the same trial can be re-run against different open-weight models by changing a single field. Every call to the model is also traced through Langfuse via request-level callbacks that propagate to the nested sub-calls of each LCEL chain. This gives a record of exactly which components were active on each trial, supporting both debugging and auditing of the reported results.

Holding the judge and metric definitions fixed across all trials is what keeps the cross-defence comparison fair and valuable, however for testing purposes our judge module is also swappable as we test different judges and their relevance to our results.

A trial is fully specified by four choices: model, attack, defences, and prompt batch. Because a trial is data rather than code, the full experimental grid is produced by iterating over configurations, which is what makes the baseline, single-defence, and defence-stack tests reproducible from the same entry point.

We implemented the selected published attacks and defences using fixed configurations so that every matrix cell could be reproduced. Most components follow the procedure described in their respective papers, although some settings were reduced to keep the complete experiment feasible on local hardware. In particular, our GCG condition uses a fixed pool of transferred and precomputed adversarial suffix candidates with black-box reranking rather than optimizing a new suffix directly against the target model.

V.II. Models

All reported target responses were generated locally through Ollama using `qwen2.5:7b-instruct`. Our original proof of concept primarily used `dolphin-mistral:7b`, which has substantially reduced safety restrictions and was useful for verifying that the evaluation pipeline worked end-to-end. We

moved to Qwen for the midterm evaluation and kept it as the target for the final experiment. Since Qwen is a safety-aligned instruction model, it provides a more realistic test of whether the attacks can bypass protections that are already present in the target model.

Several components use additional supporting models:

- PAIR [2] uses `dolphin-mistral:7b` as its attacker model and `llama3:8b` as an independent attack-time scoring model.
- GCG [1] uses `llama3:8b` as an attack-time scoring model to compare responses produced by the candidate attack prompts and select the stronger prompt.
- Perplexity filtering [3] uses `gpt2` to calculate input perplexity.
- Llama Guard [11] uses `llama-guard3:1b` to classify prompts or generated responses as safe or unsafe.
- HarmBench evaluation [8] uses `cais/HarmBench-Mistral-7b-val-cls` as the primary judge for whether a response actually fulfills the requested harmful behaviour.
- WildGuard [14] uses `allenai/wildguard` to classify whether the target response refused the request.

WildGuard replaced the local JailbreakBench refusal judge used in the midterm, which also relied on `llama3:8b`. During the preliminary evaluation, we found that the previous judge could label responses as refusals when they began with cautionary or refusal language but still went on to provide a useful answer. We therefore switched to WildGuard for the final experiment because it is designed to evaluate refusal behaviour directly. HarmBench remains separate as our harmful-behaviour judge, since whether a response refuses and whether it successfully provides the requested harmful behaviour are different outcomes.

V.III. Dataset

We evaluated 50 harmful and 50 benign prompts from JailbreakBench [7]. We chose JailbreakBench because it includes both harmful and benign prompts in a compact benchmark, making the full experiment feasible to run locally. Harmful prompts were tested across our four attack conditions and ten defence conditions, while benign prompts were evaluated without attacks under the same ten defence conditions. This produced 50 matrix cells and 2,500 final evaluated responses. This count does not include the additional model calls used internally by SmoothLLM or during PAIR and GCG attack construction.

V.IV. Attack Implementations

DeepInception [4] used the paper's nested fictional-scenario attack. Each harmful behaviour was inserted unchanged into a science-fiction scenario with five characters and five layers, culminating in a request for practical steps and summaries of the previous layers. Because the nested responses are long, DeepInception received a 2,048-token budget instead of the standard 512 tokens.

GCG [1] was implemented as a bounded black-box-reranked hybrid transfer attack instead of using gradient optimization against the target model. Calculating token gradients falls outside our project scope. For each benchmark goal, we use a fixed pool of eight pre-computed candidates: two prompt-specific artifacts originally optimized against Vicuna 13B and Llama 2 7B Chat, our earlier universal suffix, and five published Qwen2.5-1.5B universal suffixes. The target model is queried once for each candidate, and a separate `llama3:8b` goal-aware judge scores the resulting responses for compliance and fidelity. The strongest candidate is cached and reused across all defence conditions. Candidate failures are recorded while successfully scored candidates remain eligible; if no candidate can be scored, the system falls back to the source-priority candidate.

PAIR [2] was implemented as a goal-aware bounded search. We use `dolphin-mistral:7b` as the attacker model and `llama3:8b` as an independent attack-time judge. The original harmful prompt is first sent to the target and scored to establish a baseline. The attacker then generates adversarial rewrites, which are sent to the target and scored for both compliance and fidelity. The attacker receives the target's response and score as feedback to refine its next prompt. The search runs across three independent streams for up to five rounds per stream. A generated prompt only replaces the original or current best prompt when it receives a stronger score, preventing weaker rewrites from being selected simply because they were generated first. The search can use up to 16 target and judge calls per goal: one for the original prompt and up to 15 generated candidates. The strongest valid prompt is then cached and reused across all defence conditions.

V.V. Defence Implementations

Self-Reminder [9] wrapped the attacked prompt with a safety reminder before and after the user's input. The reminder tells Qwen to behave responsibly and refuse harmful requests. Since this only changes the prompt text, it does not require any additional model calls.

SmoothLLM [10] created five slightly modified copies of each attacked prompt by randomly replacing 10% of non-white-space characters. Qwen generated a response for each copy, and a simple refusal-language check grouped the responses based on whether they appeared to refuse the request. The final response was selected from the majority group using our fixed experiment seed. We used five copies to keep the runtime manageable.

Perplexity filtering [3] used `gpt2` to measure how unusual the attacked prompt was. Prompts with a perplexity above the fixed threshold of 234.74 were blocked before Qwen was called. The filter scored the attacked prompt before any Self-Reminder text was added. We kept the same threshold used in our preliminary evaluation for the final experiment.

Llama Guard [11] used `llama-guard3:1b` to classify prompts or responses as safe or unsafe. At the input stage, unsafe prompts were blocked before Qwen was called; we used this only in the Guard-sandwich stack. At the output stage, Llama Guard checked the full prompt-response exchange and replaced responses classified as unsafe with a fixed refusal.

V.VI. Combined defence stacks

The defences used in our stacks intervene at different pipeline stages: Perplexity and Llama Guard's input stage screen the prompt before generation, Self-Reminder rewrites the prompt at no extra inference cost, SmoothLLM replaces the single generation call with five perturbed-and-voted generations, and Llama Guard's output stage screens the completed ex-

change. Since execution order is fixed (input, then generation, then output), any subset composes cleanly.

Testing all 16 possible subsets was out of scope, so five stacks were chosen to isolate specific questions rather than sweep the powerset. Each reuses its components' standalone settings unchanged, so any difference from standalone behaviour reflects composition, not re-tuning.

Stack	Components	Motivating question
Self-remind. + LG-out	Self-Reminder, Llama Guard output	Can the two cheapest defences alone handle role-play (DeepInception) and refinement-search (PAIR) attacks, without Perplexity or SmoothLLM's cost?
Perplex. + SR + LG-out	Perplexity, Self-Reminder, Llama Guard output	Does adding suffix screening close the gap against GCG without SmoothLLM's five-fold generation cost?
Perplex. + Smooth + LG-out	Perplexity, SmoothLLM, Llama Guard output	Does the originally proposed trigger-oriented combination perform as expected against the full attack set?
Guard sandwich (in + out)	Llama Guard input, Llama Guard output	Do input and output moderation from the same classifier catch different failures, and what benign cost does input gating add on top of output-only gating?
Full stack	Perplexity, Self-Reminder, SmoothLLM, Llama Guard output	Does combining every layer raise the security ceiling enough to justify its cumulative latency and query cost?

Table 1: Combined-defence stacks tested in the final matrix and the design question each targets.

V.VII. Generation and Evaluation Settings

Target generation used temperature 0 and seed 42. Ordinary responses were capped at 512 tokens, while DeepInception received a 2,048-token budget because its nested prompts tend to produce longer responses. PAIR used up to five rounds across three independent streams, and SmoothLLM generated five perturbed responses per request.

HarmBench's `cais/HarmBench-Mistral-7b-val-cls` classifier [8] was used as our primary evaluator for attack success. We initially considered the official JailbreakBench behaviour judge [7], which uses Llama 3 70B, but it was too large to run practically on our local hardware and would otherwise require paid hosted inference. HarmBench's Mistral validation classifier provided a feasible local alternative, with ASR measuring whether the response actually fulfilled the requested harmful behaviour.

For refusal evaluation, we replaced the local JailbreakBench refusal judge used during the midterm with WildGuard [14]. The previous judge sometimes marked responses as refusals when they began with cautionary language but still went on to provide a useful answer. WildGuard was used to measure refusal on harmful prompts and over-refusal on benign prompts. We keep these labels separate from HarmBench ASR, since a response can avoid refusing without actually completing the harmful request.

VI. Results

The original 2,500-row experiment required approximately 6 hours and 14 minutes of recorded processing time on a local NVIDIA RTX 5080. About 4 hours and 38 minutes were spent constructing the original PAIR and GCG attacks and generating target responses across all ten defence conditions, while the WildGuard refusal and HarmBench-Mistral behaviour evaluation passes required a further 1 hour and 35 minutes. A post-run audit identified a defect in the GCG and PAIR attack-time scorer, so those 20 attack-defence cells were excluded and rerun using the corrected protocols described above. The unaffected direct-prompt, DeepInception, and benign cells were retained. The 6-hour-14-minute runtime therefore describes the original run and does not include the additional cost of the corrective rerun.

Defence	Direct	DeepInc.	GCG	PAIR
Standalone defences				
No defence	4%	66%	20%	6%
Self-reminder	2%	8%	8%	4%
SmoothLLM	6%	42%	16%	8%
Perplexity	6%	64%	**0%**	8%
Llama Guard (output)	4%	6%	4%	4%
Combined stacks				
Self-remind. + LG-out	2%	**0%**	**0%**	4%
Perplex. + SR + LG-out	2%	**0%**	**0%**	4%
Perplex. + Smooth + LG-out	6%	6%	**0%**	8%
Guard sandwich (in + out)	2%	8%	2%	**2%**
Full stack (P+SR+S+LG)	**0%**	2%	**0%**	**2%**

Lower is better; bold marks the column minimum (ties bolded); $n = 50$ per cell. LG-out = Llama Guard output stage; SR = Self-reminder; P = Perplexity; S = SmoothLLM.

Table 2: HarmBench behaviour attack success rate on harmful prompts.

As shown in Table 2, DeepInception was the strongest attack in the undefended condition, with 66% ASR. Among the standalone defences, Llama Guard output had the lowest cross-attack ASR (4.5%, 9/200) and the lowest worst-case ASR (6%); Self-Reminder was next at 5.5% (11/200) and performed best on the direct-prompt condition. The results also illustrate the limited generalization of specialized defences. Perplexity filtering blocked all 50 corrected hybrid-GCG inputs and reduced that cell to 0% ASR, but it blocked no DeepInception inputs and left ASR nearly unchanged there (64%, versus 66% without defence).

Compared with the midterm's single stack, the five combined-defence stacks generally improved the cross-attack point estimates: four had lower aggregate ASR than the best standalone defence, although Perplexity + SmoothLLM + Llama Guard output did not (5.0%, versus 4.5% for Llama Guard output). The full stack had the lowest observed cross-attack ASR, at 1.0% (2/200): 0% against direct prompts and GCG, and 2% against DeepInception and PAIR. The leaner Perplexity + Self-Reminder + Llama Guard output stack reached 1.5% (3/200), one additional HarmBench success, while using one-fifth as many response-path target calls (143 versus 715). The response-path cost comparison is discussed alongside Table 4.

Defence	Refused	Δ vs. none
No defence	6/50 (12%)	—
Self-reminder	10/50 (20%)	+8 pp
SmoothLLM	7/50 (14%)	+2 pp
Perplexity	10/50 (20%)	+8 pp
Llama Guard (output)	16/50 (32%)	+20 pp
Self-remind. + LG-out	21/50 (42%)	+30 pp
Perplex. + SR + LG-out	19/50 (38%)	+26 pp
Perplex. + Smooth + LG-out	12/50 (24%)	+12 pp
Guard sandwich (in + out)	19/50 (38%)	+26 pp
Full stack (P+SR+S+LG)	22/50 (44%)	+32 pp

WildGuard refusal / over-blocking rate on benign JBB prompts, $n = 50$ per condition; positive Δ indicates more refusals than the undefended model. Input-stage blocks are included here when WildGuard labels the returned blocked response as a refusal and are reported separately in Table 5.

Table 3: Benign refusal rate and change relative to no defence.

The final evaluator replaced the local JBB refusal heuristic with WildGuard, addressing the midterm's partial-refusal misclassification concern. Under WildGuard, the undefended model over-refused 12% of benign prompts. Every condition containing output-stage Llama Guard had a higher refusal rate, ranging from 24% to 44%. The full stack had the lowest observed ASR in the corrected results, reducing cross-attack ASR to 1.0% (2/200) but raising benign refusals to 44% (22/50), an increase of 32 percentage points over no defence. The more efficient Perplexity + Self-Reminder + Llama Guard output stack was close in ASR at 1.5% (3/200), but still raised benign refusals to 38% (19/50). These results show the central security/usability trade-off, while the small difference of one harmful success between the two lowest-ASR conditions should not be treated as evidence that one is reliably safer.

Defence	Direct	DeepInc.	GCG	PAIR	Benign
No defence	1.76	7.51	1.94	1.73	2.92
Self-reminder	1.43	4.96	1.23	1.58	2.61
SmoothLLM	9.08	34.47	9.37	10.01	14.15
Perplexity	1.74 (n=46)	7.54	blocked (n=0)	1.84 (n=47)	2.98 (n=45)
Llama Guard (output)	1.75	7.45	1.95	1.91	3.07
Self-remind. + LG-out	1.57	4.87	1.43	1.72	2.75
Perplex. + SR + LG-out	1.55 (n=46)	4.91	blocked (n=0)	1.69 (n=47)	2.74 (n=45)
Perplex. + Smooth + LG-out	9.21 (n=46)	34.64	blocked (n=0)	10.04 (n=47)	14.27 (n=45)
Guard sandwich (in + out)	3.20 (n=2)	7.73 (n=37)	3.39 (n=1)	3.21 (n=5)	3.13 (n=37)
Full stack (P+SR+S+LG)	8.26 (n=46)	26.19	blocked (n=0)	9.18 (n=47)	13.22 (n=45)

Mean latency in seconds for prompts where the target model actually executed, $n = 50$ per cell unless noted. "blocked" indicates that the input stage rejected all 50 prompts, so no non-blocked response latency exists for that cell; see Table 5. Guard-sandwich latency for Direct, GCG, and PAIR rests on only 2, 1, and 5 non-blocked observations, respectively, and should be treated as highly uncertain.

Table 4: Mean non-blocked response latency in seconds by attack and defence.

Defence	Direct	DeepInc.	GCG	PAIR	Benign
No defence	0%	0%	0%	0%	0%
Self-reminder	0%	0%	0%	0%	0%
SmoothLLM	0%	0%	0%	0%	0%
Perplexity	8%	0%	**100%**	6%	10%
Llama Guard (output)	0%	0%	0%	0%	0%
Self-remind. + LG-out	0%	0%	0%	0%	0%
Perplex. + SR + LG-out	8%	0%	**100%**	6%	10%
Perplex. + Smooth + LG-out	8%	0%	**100%**	6%	10%
Guard sandwich (in + out)	96%	26%	98%	90%	26%
Full stack (P+SR+S+LG)	8%	0%	**100%**	6%	10%

Share of prompts rejected at the input stage before target inference, $n = 50$ per cell. High values are protective on the harmful columns but represent operational cost on the Benign column. Bold marks the four Perplexity-containing conditions' identical 100% GCG block rate, confirmed at the prompt-index level to be the same 50 prompts in all four cases.

Table 5: Input-defence block rate by attack and defence.

The latency values cover the defended response path and exclude adaptive attack construction and final evaluation. SmoothLLM dominates response-path cost because it makes five target-model calls for every request that reaches it; its mean latency reaches 34.47 seconds on DeepInception, and stacks that contain it inherit most of that cost. DeepInception is slower even without SmoothLLM because its nested prompts tend to elicit longer responses and use a larger generation budget. Every Perplexity-containing condition has no non-blocked GCG latency: the frozen deterministic filter rejected all 50 corrected GCG prompts before target inference in each of those four conditions.

Across all four harmful attacks, the full stack had the lowest observed ASR, at 1.0% (2/200), compared with 1.5% (3/200) for Perplexity + Self-Reminder + Llama Guard output. This difference amounts to only one additional successful attack across 200 harmful prompts. The security difference was small, but the computational cost difference was substantial. On harmful requests that reached the target, the full stack added an average of 11.07 seconds of paired latency, whereas the three-defence stack had a paired difference of −0.99 seconds. Including the benign condition, the full stack made 940 response-path target calls, exactly five times the three-defence stack's 188. The negative latency difference for the three-defence stack should not be interpreted as the defence making inference faster; its responses simply completed sooner than the corresponding undefended responses. Overall, the full stack added substantial computational cost for only a small observed reduction in attack success.

VII. Challenges and Limitations

VII.I. Scoping of Defences

Our evaluation strictly targets black-box, inference-time wrappers. We excluded DeepAlign and JBShield because both methods require white-box access to internal model representations. Implementing them requires interventions within the model's execution, which breaks our black-box constraints. Furthermore, integrating these internal interventions into our combinatorial attack-and-defence matrix would have substantially increased the total execution time.

VII.II. Scoping of Attacks

We limited our attack implementations to DeepInception, GCG, and PAIR, excluding Tree of Attacks with Pruning (TAP) and PAPILLON. TAP functions as a branched extension of PAIR, so implementing it would have added significant search and validation cost without introducing a fundamentally different attack vector for our analysis. Similarly, PAPILLON uses automated fuzzing to generate fluent jailbreak prompts, which overlaps with the semantic and contextual attacks already represented by DeepInception and PAIR. Adding either method would also have expanded the final matrix and increased the runtime beyond our available project scope.

VII.III. Judge Deception and Secondary Attacks

A limitation of our pipeline is its vulnerability to secondary or recursive attacks. As raised during our midterm presentation, an attacker could craft an input that forces the target model to output a secondary attack specifically designed to bypass or deceive the safety judge. We did not address this vector because our evaluation harness treats the refusal judge and behaviour judge as trusted, isolated observers. Hardening the evaluators against deceptive outputs requires a multi-agent security framework, whereas our scope remains strictly focused on inference-time defences for the target model itself. As a result, our results assume that the model output is evaluated as data rather than as an adversarial prompt against the judge.

VII.IV. Hardware and Scaling Constraints

Local hardware limitations influenced our model selection and dataset size. All experiments were run locally on an NVIDIA RTX 5080. Even with the relatively small 7B and 8B models used in our pipeline, we had to carefully swap models into and out of GPU memory as different stages of the experiment ran. This made larger models, such as the 70B JailbreakBench behaviour judge, impractical within our available resources. To keep the end-to-end execution time manageable, we capped the final evaluation at 50 harmful and 50 benign prompts and tested five selected defence stacks rather than every possible combination.

VII.V. Generalization

Because we rely on a single canonical target model (`qwen2.5:7b-instruct`) to maintain a controlled experiment, our findings are heavily influenced by Qwen's specific alignment training. The observed attack success rates and defence interactions may not generalize perfectly to models with different architectures or safety-tuning approaches. Furthermore, automated evaluators such as WildGuard and HarmBench can still misclassify nuanced responses, meaning our results depend partly on the accuracy of these proxy models rather than absolute human consensus.

VII.VI. Statistical Power and Run-to-Run Variability

The evaluation uses a relatively small sample of 50 harmful and 50 benign prompts. Consequently, a small number of classification changes can noticeably affect the reported attack success and refusal rates. Model generation and adaptive attacks are also stochastic, but resource constraints prevented us from repeating every configuration across multiple random seeds. Our results should therefore be interpreted as comparative trends within this experimental setup rather than precise estimates of real-world performance.

VII.VII. Attack Budget and Implementation Sensitivity

Adaptive attack performance depends heavily on implementation choices, including iteration limits, generation temperature, stopping criteria, attacker models, and query budgets. Although we attempted to apply consistent budgets, equal query counts do not necessarily represent equal attack strength across GCG, PAIR, and DeepInception. Different hyperparameter choices or longer optimization runs could change both absolute success rates and the relative ranking of the defences.

VIII. Future Directions

One natural extension is to broaden the threat model beyond black-box inference. Our study deliberately leaves the target model's weights and hidden representations unchanged, which excludes both target-optimized GCG [1] and representation-level defences such as JBSHIELD [12] and DEEPALIGN [13]. A white-box extension could evaluate these methods in a separate experimental track and ask whether access to model internals changes the security–usability–cost trade-off observed for external inference-time wrappers. More importantly, it could test whether internal interventions and lightweight external defences provide complementary protection or simply defend against the same failures through different mechanisms.

A second direction is to move from defence-oblivious attacks to defence-aware adversaries. In the current experiment, adaptive attack prompts are constructed once and reused across defence conditions so that comparisons remain paired. A stronger attacker could instead observe the deployed defence stack and optimize directly against it. Larger-budget PAIR searches [2], Tree of Attacks with Pruning (TAP) [5], and fuzz-testing approaches such as PAPILLON [6] provide natural starting points. This raises a more difficult robustness question: does the relative ranking of defences remain stable when the attacker is allowed to adapt to the defence itself?

Future work could also replace fixed defence stacks with conditional defence policies. The five stacks evaluated here were selected to test specific hypotheses rather than exhaust every possible combination. An adaptive system could begin with inexpensive mechanisms such as Self-Reminder or Perplexity and invoke additional classifiers or SmoothLLM only when earlier stages indicate elevated risk. Such a system would shift the question from "which fixed stack is best?" to "when is each defence worth invoking?" A useful objective would be to determine whether conditional routing can approach the security of a large stack while retaining the latency, target-call cost, and benign usability of a smaller one.

Finally, the experimental design should be replicated across model families, scales, and evaluator choices. Our controlled study uses one canonical Qwen target and locally feasible model sizes; additional compute would allow the same frozen attack–defence matrix to be evaluated on models with different architectures and alignment strategies. Human adjudication and alternative safety judges could also be used to measure how sensitive conclusions are to WildGuard and HarmBench. A particularly important extension is to treat the

evaluators themselves as part of the attack surface: adversarial outputs could be constructed specifically to mislead refusal or behaviour classifiers, allowing future work to study whether defence conclusions remain valid when the judges are no longer assumed to be trusted observers.

IX. Conclusion

This work evaluated whether inference-time jailbreak defences generalize across attack families, whether defence-in-depth improves on the strongest individual layer, and what those gains cost in usability and computation. Across four attack conditions, ten defence conditions, and 2,500 evaluated responses, three main conclusions emerge.

First, jailbreak defences do not generalize uniformly. Perplexity filtering blocked all tested GCG inputs but provided little protection against the fluent DeepInception attack, while Self-Reminder and Llama Guard were more consistent across attack types. Strong performance against one jailbreak family therefore does not imply broader robustness.

Second, defence-in-depth is most useful when its layers provide complementary protection. The full stack achieved the lowest observed cross-attack ASR at 1.0% (2/200), while the leaner Perplexity + Self-Reminder + Llama Guard output stack reached 1.5% (3/200). This difference amounts to only one additional successful attack across 200 harmful prompts, while the full stack required substantially more target-model calls and incurred much higher SmoothLLM-driven latency. Adding every available layer therefore produced only a small additional reduction in attack success at a much larger computational cost.

Finally, stronger protection came with a clear usability cost. The three-defence stack increased benign refusal from 12% without defence to 38%, while the full stack reached 44%. Minimizing attack success alone therefore does not identify the most practical defence.

Overall, our results suggest that inference-time jailbreak defence should be treated as a multi-objective systems problem rather than a search for the strongest individual filter or largest stack. Under the tested Qwen target and JailbreakBench prompts, a smaller combination of complementary mechanisms came close to the full stack's observed security while requiring substantially less computation, but substantial benign over-refusal remained. Robustness across attack families, benign usability, and deployment cost should therefore be evaluated together.

X. References

[1] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, "Universal and Transferable Adversarial Attacks on Aligned Language Models (GCG)." [Online]. Available: https://arxiv.org/abs/2307.15043

[2] P. Chao, A. Robey, E. Dobriban, H. Hassani, G. J. Pappas, and E. Wong, "Jailbreaking Black Box Large Language Models in Twenty Queries (PAIR)." [Online]. Available: https://arxiv.org/abs/2310.08419

[3] G. Alon and M. Kamfonas, "Detecting Language Model Attacks with Perplexity." [Online]. Available: https://arxiv.org/abs/2308.14132

[4] X. Li, Z. Zhou, J. Zhu, J. Yao, T. Liu, and B. Han, "DeepInception: Hypnotize Large Language Model to Be Jailbreaker." [Online]. Available: https://arxiv.org/abs/2311.03191

[5] A. Mehrotra *et al.*, "Tree of Attacks: Jailbreaking Black-Box LLMs Automatically," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. [Online]. Available: https://arxiv.org/abs/2312.02119

[6] X. Gong *et al.*, "PAPILLON: Efficient and Stealthy Fuzz Testing-Powered Jailbreaks for LLMs," in *Proceedings of the 34th USENIX Security Symposium*, 2025.

[7] P. Chao *et al.*, "JailbreakBench: An Open Robustness Benchmark for Jailbreaking Large Language Models," in *Advances in Neural Information Processing Systems (NeurIPS) Track on Datasets and Benchmarks*, 2024. [Online]. Available: https://arxiv.org/abs/2404.01318

[8] M. Mazeika *et al.*, "HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal." [Online]. Available: https://arxiv.org/abs/2402.04249

[9] Y. Xie *et al.*, "Defending ChatGPT against Jailbreak Attack via Self-Reminders," *Nature Machine Intelligence*, vol. 5, pp. 1486–1496, 2023, doi: 10.1038/s42256-023-00765-8.

[10] A. Robey, E. Wong, H. Hassani, and G. J. Pappas, "SmoothLLM: Defending Large Language Models Against Jailbreaking Attacks." [Online]. Available: https://arxiv.org/abs/2310.03684

[11] H. Inan *et al.*, "Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations." [Online]. Available: https://arxiv.org/abs/2312.06674

[12] S. Zhang *et al.*, "JBShield: Defending Large Language Models from Jailbreak Attacks through Activated Concept Analysis and Manipulation," in *Proceedings of the 34th USENIX Security Symposium*, 2025.

[13] Y. Zhang, T. Liu, Z. Zhao, G. Meng, and K. Chen, "Bleeding Pathways: Vanishing Discriminability in LLM Hidden States Fuels Jailbreak Attacks," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2025.

[14] S. Han *et al.*, "WildGuard: Open One-Stop Moderation Tools for Safety Risks, Jailbreaks, and Refusals of LLMs," in *Advances in Neural Information Processing Systems (NeurIPS) Track on Datasets and Benchmarks*, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/0f69b4b96a46f284b726fbd70f74fb3b-Abstract-Datasets_and_Benchmarks_Track.html